INSTALLATION & CODE CONVERSION MANUAL

Legacy PHP Application

Running a PHP 4 / 5 codebase on XAMPP or WAMP, and migrating it to modern PHP 8

Two supported paths: (A) run unchanged on a legacy stack • (B) modernise the code for PHP 8

Document type: Step-by-step technical manual
Audience: Developers and administrators deploying the application locally

Prepared by

Dr. Prakashgoud R. Patil
Professor and Head
Department of Computer Applications
KLE Technological University, Hubballi
Email: prpatilji@gmail.com, prakashpatil@kletech.ac.in

Contents

1. Overview

This manual explains how to set up and run a legacy PHP application on a Windows machine using either XAMPP or WAMP, and how to migrate the same code so it runs safely on a current, supported version of PHP.

The guidance is based on the syntax observed in the source code. A call such as the one below is a strong indicator that the project was written for an older PHP release:

```
retrievesupplier($conn, $BookSupplierID, &$BookSupplierName, ...);
```

The **&** placed before an argument in the function *call* is known as **call-time pass-by-reference**. This feature was deprecated in PHP 5.3 and **removed entirely in PHP 5.4**, so the code most likely targets PHP 5.2 or earlier.

You have two supported options. Read Section 3 to decide which one fits your situation, then follow the matching procedure.

> **How to use this manual**
> Section 2 – confirm which PHP version the code targets.
> Section 3 – choose Path A (run unchanged) or Path B (modernise).
> Section 4 – install a legacy XAMPP / WAMP stack (Path A).
> Section 5 – install modern PHP and convert the deprecated code (Path B, recommended).
> Section 6 – quick reference of every deprecated feature and its replacement.

2. Step 1 – Identify the PHP version the code targets

Before installing anything, scan the source code for the markers below. The more of these you find, the older the target PHP version, and the stronger the case for using PHP 5.2 rather than 5.3.

2.1 Tell-tale signs of legacy code

- `mysql_connect()` – old MySQL extension (removed in PHP 7).
- `mysql_query()` – old MySQL query function (removed in PHP 7).
- `session_register()` – obsolete session handling (removed in PHP 5.4).
- `split()` – POSIX string split (removed in PHP 7).
- `ereg()` / `eregi()` – POSIX regular expressions (removed in PHP 7).
- **&** in function calls – call-time pass-by-reference (removed in PHP 5.4).
- `register_globals` – reliance on auto-created globals (removed in PHP 5.4).

2.2 Quick procedure

1. Open the project folder and search all .php files for the keywords listed above (any code editor's “Find in Files” works).

2. If you find `&` in function calls, `session_register()`, or `register_globals`, treat the project as PHP 5.2.
3. If you only find `mysql_*` calls, the project will usually run on PHP 5.3 as well, but PHP 5.2 remains the safest target.
4. Use this conclusion to pick the correct stack version in Section 4, or proceed to Section 5 to modernise.

3. Step 2 – Choose your approach

There are two supported paths. The right choice depends on whether you are allowed to change the source code and whether the application must remain online securely.

Path A – Run the code unchanged on a legacy stack

Install an old XAMPP or WAMP version that ships with PHP 5.2 / 5.3. No code changes are required. Best when you simply need to view or run the application briefly and cannot modify it.

Path B – Modernise the code for PHP 8 (recommended)

Install a current XAMPP / WAMP (PHP 8.x) and make a small set of compatibility edits – mainly removing the & from function calls and replacing a handful of removed functions. This is the safer, supported, long-term option and usually takes only a few minutes.

> **Recommendation**
> Path A is convenient but runs on PHP that is 15+ years old and no longer receives security updates – use it only for short-lived, offline testing.
> Path B is preferred for anything that will stay in use. The required code changes are small and well understood, and you gain a secure, supported runtime.

3.1 Decision guide

If…	Then choose
You cannot change the source code at all	**Path A – legacy stack (PHP 5.2 / 5.3)**
You need the app online and secure	**Path B – modernise for PHP 8**
You are on Windows 10 / 11 and old installers fail	**Path B – modernise for PHP 8**
You only need a quick, offline look at the app	**Path A – legacy stack**
You plan to maintain or extend the app	**Path B – modernise for PHP 8**

4. Path A – Run without code changes (legacy stack)

Use this path only if you must run the application exactly as-is. Pick either XAMPP or WAMP – you do not need both.

4.1 Option 1 – XAMPP (recommended for legacy)

XAMPP legacy installers are generally easier to find and tend to be more stable for old PHP applications. Choose the version that matches your target PHP.

XAMPP version	PHP version	Compatibility
XAMPP 1.7.1	PHP 5.2.9	**Best** Best choice – last release before most 5.3 deprecations.
XAMPP 1.7.2	PHP 5.3.0	**Good** Good choice.
XAMPP 1.7.3	PHP 5.3.x	**Good** Good choice.
XAMPP 1.7.4	PHP 5.3.5	**Warn** Works, but may show deprecation warnings.
XAMPP 1.7.7	PHP 5.3.6 / 5.3.8	**Warn** Usually works; deprecated features may warn.

If the project uses `mysql_connect()`, `session_register()`, `split()`, `ereg()`, or `&` in function calls, the safest choice is **XAMPP 1.7.1 (PHP 5.2.9)**.

Installation steps (XAMPP)

1. Download the chosen legacy XAMPP installer (see Section 4.3 for sources).
2. Run the installer and install to a simple path such as `C:\xampp`. Avoid installing into Program Files.
3. Open the XAMPP Control Panel and start the Apache and MySQL modules.
4. Copy your project folder into `C:\xampp\htdocs\` (for example `C:\xampp\htdocs\myapp`).
5. If the app uses a database, open `http://localhost/phpmyadmin` and import the project's SQL dump; then confirm the DB name, user and password in the app's config file.
6. Browse to `http://localhost/myapp/` to run the application.

4.2 Option 2 – WAMP

WampServer is an alternative if you prefer it. Match the WAMP version to your target PHP as shown below.

WAMP version	PHP version	Compatibility
WampServer 2.0i	PHP 5.2.x	**Best** Best choice for PHP 5.2 code.
WampServer 2.1	PHP 5.3.x	**OK** Usually works; may show deprecation warnings.

WAMP version	PHP version	Compatibility
WampServer 2.2	PHP 5.3.8	**OK** Usually works; deprecated features may warn.
WampServer 2.4+	PHP 5.4+	**No** Not compatible with call-time pass-by-reference.
WampServer 3.x	PHP 7.x / 8.x	**No** Not compatible without code changes (use Path B).

Recommended: WampServer 2.0i (PHP 5.2.x), or **WampServer 2.1 (PHP 5.3.x)**. These are the versions most likely to execute the legacy syntax successfully.

Installation steps (WAMP)

1. Download the chosen legacy WampServer installer (see Section 4.3 for sources).
2. Install to a simple path such as `C:\wamp`, then launch WampServer and wait for the tray icon to turn green.
3. Copy your project folder into `C:\wamp\www\`.
4. Use the bundled phpMyAdmin to create the database and import the SQL dump, then update the app's database credentials.
5. Browse to `http://localhost/myapp/` to run the application.

4.3 Finding legacy installers

Old XAMPP and WAMP releases are no longer hosted on the official download pages, which only carry current versions. Legacy builds can usually be found at archive sites such as:

- **SourceForge** – the official WampServer project area still lists many older releases.
- **Apache Friends / XAMPP archives** and third-party archive mirrors that retain old installers.

Because exact archive URLs change over time, locate the installer by searching for the specific version (for example, “XAMPP 1.7.1 download”) on a reputable archive, and verify the file before running it.

> **Caution on Windows 10 / 11**
> These legacy XAMPP and WAMP builds are over 15 years old and may not run reliably on modern Windows.
> If an old installer fails to start, refuses to install, or Apache will not launch, switch to Path B: install a current stack (PHP 8.x) and make the small compatibility edits in Section 5.

5. Path B – Run on modern PHP (recommended)

This path installs a current, supported stack and applies a small set of compatibility edits. For most legacy projects the changes are minor and can be completed in a few minutes.

5.1 Install a modern stack

1. Install a current **XAMPP 8.x** or **WampServer 3.x** from the official site.
2. Place the project under the web root (`htdocs` for XAMPP, `www` for WAMP) and start Apache + MySQL.
3. Load the app once in the browser and note the errors PHP reports – these point directly to the lines that need the edits in Section 5.3.
4. Apply the conversions below, then reload until the application runs cleanly.

5.2 What needs to change

Modern PHP removed several features the legacy code relies on. The most common change is removing the & from function calls; the rest are simple function replacements.

Deprecated / removed	Modern replacement	Notes
`func(&$arg)  // at call`	`func($arg)  // at call`	Remove & from the call; declare the reference in the function signature instead. Removed in PHP 5.4.
`mysql_connect()`	`mysqli_connect() / PDO`	mysql_* removed in PHP 7. Use MySQLi or PDO.
`mysql_query()`	`mysqli_query() / PDO::query`	Pass the connection handle as an argument.
`mysql_fetch_array()`	`mysqli_fetch_array()`	Same idea across all mysql_* fetch calls.
`session_register()`	`$_SESSION['x'] = $x;`	Use the $_SESSION superglobal. Removed in 5.4.
`split()`	`explode() / preg_split()`	explode() for fixed strings; preg_split() for regex. Removed in PHP 7.
`ereg() / eregi()`	`preg_match()`	Wrap the pattern in delimiters; add /i for eregi(). Removed in PHP 7.
`register_globals`	`$_GET / $_POST / $_SESSION`	Reference request data via superglobals. Removed in 5.4.

5.3 Conversion steps with examples

Step 1 – Remove & from function calls (the main fix)

Delete the ampersand in front of arguments at the call site. If the function genuinely needs to modify that argument, declare the reference in the function definition instead.

Before

```
// Call site (legacy) – ampersand before the argument
retrievesupplier($conn, $BookSupplierID, &$BookSupplierName, &$BookSupplierAddr);
```

After

```
// Call site – no ampersand
retrievesupplier($conn, $BookSupplierID, $BookSupplierName, $BookSupplierAddr);

// Function definition – declare the reference here instead (& on the parameter)
function retrievesupplier($conn, $BookSupplierID, &$BookSupplierName,
&$BookSupplierAddr) {
    // ... populates $BookSupplierName and $BookSupplierAddr by reference
}
```

Step 2 – Replace the old MySQL extension

Migrate mysql_* calls to MySQLi (or PDO). Note that MySQLi takes the connection handle as a parameter.

Before

```
$conn = mysql_connect('localhost', 'root', '');
mysql_select_db('mydb');
$result = mysql_query("SELECT * FROM suppliers");
$row = mysql_fetch_array($result);
```

After

```
$conn = mysqli_connect('localhost', 'root', '', 'mydb');
$result = mysqli_query($conn, "SELECT * FROM suppliers");
$row = mysqli_fetch_array($result);
```

Step 3 – Replace POSIX regex and split()

Before

```
$parts = split(',', $csvLine);
if (ereg('^[0-9]+$', $value)) { /* ... */ }
```

After

```
$parts = explode(',', $csvLine);          // fixed delimiter
// $parts = preg_split('/\s*,\s*/', $csvLine);  // if a regex is needed
if (preg_match('/^[0-9]+$/', $value)) { /* ... */ }
```

Step 4 – Replace session_register() and register_globals usage

Before

```
session_register('username');
$username = 'alice';   // relied on register_globals
```

After

```
session_start();
$_SESSION['username'] = 'alice';
// Read request data explicitly via superglobals:
$id = isset($_GET['id']) ? $_GET['id'] : null;
```

5.4 Verify the migration

1. During testing, enable error reporting at the top of the entry script so nothing is hidden: `error_reporting(E_ALL); ini_set('display_errors', 1);`
2. Exercise each major feature (login, listings, forms, database writes) and fix any remaining warnings the same way.
3. Once the app runs cleanly with no deprecation or fatal errors, switch error display off for any non-local use.
4. For better security, progressively move database access to **prepared statements** (MySQLi or PDO) to protect against SQL injection.

6. Deprecated features – quick reference

Use this table as a checklist while editing. Anything in the left column should be replaced when moving to PHP 7 / 8.

Feature	Removed in	Replace with
`Call-time pass-by-reference func(&$x)`	**PHP 5.4**	Remove & at call; declare &$x in the function definition
`session_register()`	**PHP 5.4**	$_SESSION['x'] = ...
`register_globals`	**PHP 5.4**	$_GET / $_POST / $_REQUEST / $_SESSION
`magic_quotes_gpc`	**PHP 5.4**	Manual escaping / prepared statements
`mysql_* functions`	**PHP 7.0**	mysqli_* or PDO
`split() / spliti()`	**PHP 7.0**	explode() or preg_split()
`ereg() / eregi()`	**PHP 7.0**	preg_match() (add /i for eregi)

Bottom line

Need it running quickly and unchanged → Path A with XAMPP 1.7.1 (PHP 5.2.9), or WampServer 2.0i.

Need it secure and maintainable → Path B: install PHP 8.x, remove & from function calls, and replace the handful of removed functions listed above.

For an exact version match, inspecting the full project (rather than a single line) gives the most accurate answer.